



Proyecto Final: Animación de Figura Humanoide

Guillermo Luis Osuna González
Álgebra Geométrica Conformal

Equipo: Las Chivas
Omar Alejandro Sánchez López
Omar Alejandro Santos Ibarra
Carlos Santiago Cruz Díaz
27/05/2026

Índice:

Índice:	1
Introducción.....	2
Contenido	3
Conclusiones.....	21
Bibliografía:.....	23



Introducción

En el ámbito de la computación gráfica y la animación digital, la representación y el control cinemático de estructuras articuladas complejas —como el cuerpo humano— representan un desafío computacional clásico. Tradicionalmente, los sistemas de simulación han dependido del álgebra matricial lineal y de parametrizaciones como los ángulos de Euler o las matrices de transformación homogénea. Sin embargo, estas aproximaciones convencionales arrastran limitaciones intrínsecas notorias, tales como la alta carga computacional en operaciones de rotación-traslación acopladas.

Para superar estas limitantes, el presente proyecto aborda la modelación y simulación cinemática de una estructura humanoide tridimensional de 32 grados de libertad (DOF) utilizando como fundamento el Álgebra Geométrica Conformal (CGA), operada bajo el entorno de desarrollo CLUViz. Concebida como una extensión geométrica basada en el espacio de Minkowski, la CGA unifica en un solo marco algebraico elegante los puntos, líneas, planos y esferas, permitiendo modelar transformaciones rígidas globales (rotaciones y traslaciones) de forma unificada mediante el uso de motores conformales.

El propósito medular de este trabajo es desarrollar un sistema fiel capaz de alternar dinámicamente entre dos modos operativos críticos para la ingeniería de la animación: un modo manual orientado al control individual e interactivo de cada articulación mediante restricciones angulares anatómicas, y un modo animación automatizado, programado para decodificar, procesar e interpolar de forma fluida matrices de movimiento procedentes de un archivo de datos estructurado. A lo largo de este informe se detalla la arquitectura de la cadena cinemática directa implementada, las estrategias matemáticas adoptadas para la estabilización y normalización geométrica de los componentes volumétricos del esqueleto, y las soluciones aplicadas para optimizar el rendimiento del renderizador gráfico, logrando un entorno de simulación autónomo, robusto y matemáticamente exacto.

Contenido

1. Arquitectura y Estructura del Script Dinámico

El software desarrollado en CLUViz implementa una simulación cinemática completa de un modelo humanoide articulado de 32 grados de libertad (DOF), utilizando como marco teórico el **Álgebra Geométrica Conformal (CGA)** en un espacio de Minkowski.

Con la finalidad de garantizar la legibilidad, mantenibilidad y modularidad del código, el programa se estructuró dividiendo el script principal de ejecución de las macros visuales y de carga matemática. La jerarquía del algoritmo sigue un orden estricto de ejecución:

1. **Inicialización Conformal:** Definición de los generadores $e1$, $e2$ y $e3$, el origen $e0$ y el punto en el infinito $einf$ para parametrizar el espacio de representación rígida.
2. **Declaración de Variables Antropométricas:** Configuración de la escala métrica real y longitudes nativas del esqueleto según los parámetros de diseño especificados.
3. **Maapeo Cinematográfico:** Implementación de la cadena secuencial de motores y rotores para calcular la posición en el espacio conformal de cada articulación.
4. **Capa de Renderizado Anatómico:** Dibujo secuencial de los volúmenes del personaje y del escenario perimetral interactivo.

Dichas secciones son apreciables en el código implementado:

```
DefVarsN3();
```

```
DrawFrame(0.5,"axes");
```

```
/**
```

```
//Funcion cilindro modificada
```

```
Cilindro = {
```

```
    //Entrada de datos: _P(#)
```

```
    A = _P(1); //Entrada 1: Punto inicial del cilindro
```

```
    B = _P(2); //Entrada 2: Punto final del cilindro
```

```
    r = _P(3); //Entrada 3: Radio del cilindro
```

```
    E = einf^e0; //Plano de Minkowski
```

```
    Ae = (A^E)*E; //Quitar parte "conformal" a Punto A
```

```

Be = (B^E)*E; //Quitar parte "conformal" a Punto B
Ce = Be - Ae; //Vector de direccion en Euclidiano que va de A a B
C = Ce + 1/2*Ce^2*einf + e0; // Vector Ce en conformal
DrawCylinder(A,C,r);
}

```

```

Box = {
    C = _P(1); // Entrada 1: Punto centro de la caja
    X = _P(2); // Entrada 2: Punto direccion 1
    Y = _P(3); // Entrada 3: Punto direccion 2
    Z = _P(4); // Entrada 4: Punto direccion 3

```

```

    E = einf^e0; // Plano de Minkowski
    D1e = ((X-C)^E)*E;
    D2e = ((Y-C)^E)*E;
    D3e = ((Z-C)^E)*E;
    D1 = D1e + 1/2*D1e^2*einf + e0;
    D2 = D2e + 1/2*D2e^2*einf + e0;
    D3 = D3e + 1/2*D3e^2*einf + e0;
    DrawBox(C,D1,D2,D3);
}

```

```

:IPNS;

```

```

check = CheckBox("Animar",false);

```

```

EnableAnimate(check);

```

```

if (ExecMode & !EM_ANIMATE) {
    Data = ReadData("datos.dat");
    ?N = Size(Data);
}

```

```
if(check){  
//Numero de elementos de la lista  
    ?i = floor(Time*200)%N+1;
```

```
//////////
```

```
//Movimiento general  
    BFx = Data(i,1);  
    BFy = Data(i,2);  
    BFz = Data(i,3);  
    BFRx = Data(i,4);  
    BFRy = Data(i,5);  
    BFRz = Data(i,6);
```

```
//Columna  
    LLJxr = Data(i,7);  
    LLJzr = Data(i,8);
```

```
//Cuello  
    LNJxr = Data(i,9);  
    LNJyr = Data(i,10);
```

```
//Hombro izquierdo  
    SJLxr = Data(i,11);  
    SJLyr = Data(i,12);  
    SJLzr = Data(i,13);
```

```
//Codo izquierdo  
    EJLxr = Data(i,14);
```

```
//Hombro derecho  
    SJRxr = Data(i,15);
```

```
SJRyr = Data(i,16);
SJRzr = Data(i,17);

//Codo derecho
EJRxr = Data(i,18);

//Cadera izquierda
HJLxr = Data(i,19);
HJLyr = Data(i,20);
HJLzr = Data(i,21);

//Rodilla izquierda
KJLxr = Data(i,22);

//Tobillo izquierdo
AJLxr = Data(i,23);
AJLyr = Data(i,24);
AJLzr = Data(i,25);

//Cadera derecha
HJRxr = Data(i,26);
HJRyr = Data(i,27);
HJRzr = Data(i,28);

//Rodilla derecha
KJRxr = Data(i,29);

//Tobillo derecho
AJRxr = Data(i,30);
AJRyr = Data(i,31);
AJRzr = Data(i,32);
```

```

} else {
//////////
    //Movimiento general
    BFX = Slider("Body Forward X", -5,5,0.1,0);
    BFY = Slider("Body Forward Y", -5,5,0.1,0.93);
    BFZ = Slider("Body Forward Z", -5,5,0.1,0);
    BFRX = Slider("Body Rotation X", -Pi,Pi,0.1,0);
    BFRY = Slider("Body Rotation Y", -Pi,Pi,0.1,0);
    BFRZ = Slider("Body Rotation Z", -Pi,Pi,0.1,0);

    //Columna
    LLJxr = Slider("Lumbar Rotation X", -0.6,0.6,0.1,0);
    LLJzr = Slider("Lumbar Rotation Z", -0.3,0.3,0.1,0);

    //Cuello
    LNJxr = Slider("Neck Rotation X", -0.6,0.6,0.1,0);
    LNJyr = Slider("Neck Rotation Y", -Pi/4,Pi/4,0.1,0);

    //Hombro izquierdo
    SJLxr = Slider("Left Shoulder Rotation X", -Pi,2,0.1,0);
    SJLyr = Slider("Left Shoulder Rotation Y", -Pi/4,Pi/4,0.1,0);
    SJLzr = Slider("Left Shoulder Rotation Z", -Pi,Pi/2,0.1,0);

    //Codo izquierdo
    EJLxr = Slider("Left Elbow Rotation X", -Pi,0,0.1,0);

    //Hombro derecho
    SJRxr = Slider("Right Shoulder Rotation X", -Pi,2,0.1,0);
    SJRyr = Slider("Right Shoulder Rotation Y", -Pi/4,Pi/4,0.1,0);
    SJRzr = Slider("Right Shoulder Rotation Z", Pi,-Pi/2,0.1,0);

```



```

//Codo derecho
    EJRxr = Slider("Right Elbow Rotation X", -Pi,0,0.1,0);

//Cadera izquierda
    HJLxr = Slider("Left Hip Rotation X", -Pi/1.5,Pi/1.5,0.1,0);
    HJLyr = Slider("Left Hip Rotation Y", -Pi/2,Pi/2,0.1,0);
    HJLzr = Slider("Left Hip Rotation Z", -Pi/3,Pi/3,0.1,0);

//Rodilla izquierda
    KJLxr = Slider("Left Knee Rotation X", 0,Pi,0.1,0);

//Tobillo izquierdo
    AJLxr = Slider("Left Anckle Rotation X", -Pi/3,Pi/3,0.1,0);
    AJLyr = Slider("Left Anckle Rotation Y", -Pi/2,Pi/2,0.1,0);
    AJLzr = Slider("Left Anckle Rotation Z", -Pi/2,Pi/2,0.1,0);

//Cadera derecha
    HJRxr = Slider("Right Hip Rotation X", -Pi/1.5,Pi/1.5,0.1,0);
    HJRyr = Slider("Right Hip Rotation Y", -Pi/2,Pi/2,0.1,0);
    HJRzr = Slider("Right Hip Rotation Z", -Pi/3,Pi/3,0.1,0);

//Rodilla derecha
    KJRxr = Slider("Right Knee Rotation X", 0,Pi,0.1,0);

//Tobillo derecho
    AJRxr = Slider("Right Anckle Rotation X", -Pi/3,Pi/3,0.1,0);
    AJRyr = Slider("Right Anckle Rotation Y", -Pi/2,Pi/2,0.1,0);
    AJRzr = Slider("Right Anckle Rotation Z", -Pi/2,Pi/2,0.1,0);
}
//////////

```

//Motor global

Mg

=

TranslatorN3(BFx,BFy,BFz)*RotorN3(1,0,0,BFRx)*RotorN3(0,1,0,BFRy)*RotorN3(0,0,1,
BFRz);

M = Mg; //motor auxiliar "acumulado"

//////////

// Cuerpo

x = 0.005; // Escala 1 = mm

Ptorso = M*e0*(~M); //Punto inicial (origen)

// =====

// MEDIDAS HUMANOIDE

// =====

Lt = 409.4*x; // torso

Ln = 120*x; // cuello

Ls = 176.3*x; // medio ancho hombros

La = 239.3*x; // brazo

Lf = 250*x; // antebrazo

Lh = 186.7*x; // medio ancho cadera

Hh = 119.8*x; // altura cadera

Lp = 380.6*x; // muslo

Lc = 358.6*x; // pantorrilla

Sk = (207.9-186.7)*x; // separación rodillas

Sa = (211.6-186.7)*x; // separación tobillos

Hf = 71.8*x; // altura pie

// =====
// TRONCO Y EXTREMIDADES SUPERIORES
// =====

// Movimiento lumbar

Rlumbar = RotorN3(1,0,0,LLJxr)*RotorN3(0,0,1,LLJzr);

Tlumbar = TranslatorN3(0,Lt,0);

Mlumbar = Rlumbar * Tlumbar;

Mtorso = M * Mlumbar;

Pchest = Mtorso * e0 * (~Mtorso);

// Movimiento cuello

Rneck = RotorN3(1,0,0,LNJxr)*RotorN3(0,1,0,LNJyr);

Tneck = TranslatorN3(0,Ln,0);

Mneck = Mtorso * Rneck * Tneck;

Phead = Mneck * e0 * (~Mneck);

// Movimiento brazo izquierdo

// Hombro izquierdo

TshoulderL = TranslatorN3(-Ls, 0, 0);

RshoulderL = RotorN3(1,0,0,SJLxr)*RotorN3(0,1,0,SJLyr)*RotorN3(0,0,1,SJLzr);

TarmL = TranslatorN3(0,-La,0);

MarmL = Mtorso * TshoulderL * RshoulderL * TarmL;

PshoulderL = (Mtorso * TshoulderL) * e0 * (~(Mtorso * TshoulderL));

PelbowL = MarmL * e0 * (~MarmL);

// Codo izquierdo

RelbowL = RotorN3(1,0,0,EJLxr);

```

TforearmL = TranslatorN3(0,-Lf,0);
MforearmL = MarmL * RelbowL * TforearmL;
PhandL = MforearmL * e0 * (~MforearmL);

// Movimiento brazo derecho
// Hombro derecho
TshoulderR = TranslatorN3(Ls, 0, 0);
RshoulderR = RotorN3(1,0,0,SJRxr)*RotorN3(0,1,0,SJRyr)*RotorN3(0,0,1,SJRzr);
TarmR = TranslatorN3(0,-La,0);
MarmR = Mtorso * TshoulderR * RshoulderR * TarmR;
PshoulderR = (Mtorso * TshoulderR) * e0 * (~(Mtorso * TshoulderR));
PelbowR = MarmR * e0 * (~MarmR);

// Codo derecho
RelbowR = RotorN3(1,0,0,EJRxr);
TforearmR = TranslatorN3(0,-Lf,0);
MforearmR = MarmR * RelbowR * TforearmR;
PhandR = MforearmR * e0 * (~MforearmR);

// =====
// CADERA Y EXTREMIDADES INFERIORES
// =====

ThipCenter = TranslatorN3(0, -Hh, 0);
MhipCenter = M * ThipCenter;

// Cadera Izquierda
ThipL = TranslatorN3(-(Lh/2), 0, 0);
RhipL = RotorN3(1,0,0,HJLxr)*RotorN3(0,1,0,HJLyr)*RotorN3(0,0,1,HJLzr);
PhipL = (MhipCenter * ThipL) * e0 * (~(MhipCenter * ThipL));

```

// Cadera Derecha

ThipR = TranslatorN3((Lh/2), 0, 0);

RhipR = RotorN3(1,0,0,HJRxr)*RotorN3(0,1,0,HJRyr)*RotorN3(0,0,1,HJRzr);

PhipR = (MhipCenter * ThipR) * e0 * (~(MhipCenter * ThipR));

// Pierna izquierda

TlegL = TranslatorN3(0, -Lp, 0);

MlegL = MhipCenter * ThipL * RhipL * TlegL;

PkneeL = MlegL * e0 * (~MlegL);

// Rodilla izquierda

RkneeL = RotorN3(1,0,0,KJLxr);

TcalfL = TranslatorN3(0, -Lc, 0);

McalfL = MlegL * RkneeL * TcalfL;

PankleL = McalfL * e0 * (~McalfL);

// Tobillo Izquierdo

RankleL = RotorN3(1,0,0,AJLxr)*RotorN3(0,1,0,AJLyr)*RotorN3(0,0,1,AJLzr);

TfootL = TranslatorN3(0, -Hf, 50*x); // 50*x proyecta el empeine hacia el frente en Z

MfootL = McalfL * RankleL * TfootL;

PfootL = MfootL * e0 * (~MfootL);

// Pierna derecha

TlegR = TranslatorN3(0, -Lp, 0);

MlegR = MhipCenter * ThipR * RhipR * TlegR;

PkneeR = MlegR * e0 * (~MlegR);

// Rodilla Derecha

RkneeR = RotorN3(1,0,0,KJRxr);

TcalfR = TranslatorN3(0, -Lc, 0);

McalfR = MlegR * RkneeR * TcalfR;

PankleR = McalfR * e0 * (~McalfR);

// Tobillo Derecho

RankleR = RotorN3(1,0,0,AJRxr)*RotorN3(0,1,0,AJRyr)*RotorN3(0,0,1,AJRzr);

TfootR = TranslatorN3(0, -Hf, 50*x);

MfootR = McalfR * RankleR * TfootR;

PfootR = MfootR * e0 * (~MfootR);

// =====

// DIBUJO DEL CUERPO HUMANOIDE REALISTA

// =====

// --- 1. CABEZA Y CUELLO ---

:Color(0.85, 0.75, 0.65); // Tono de piel

// Cuello (Conecta el pecho con la base de la cabeza)

Cilindro(Pchest, Phead, 0.1);

// Cabeza

DrawSphere(Phead, 0.3);

// --- 2. TORSO Y CADERA ---

:Color(0.2, 0.4, 0.8); // Color de la "playera"

// Columna central

Cilindro(Ptorso, Pchest, 0.2);

// Clavicula

Cilindro(PshoulderL, PshoulderR, 0.12);

:Color(0.1, 0.2, 0.5); // Color del "pantalón"

// Triangulo de la cadera

Cilindro(Ptorso, PhipL, 0.15); // Lado izquierdo de la pelvis

Cilindro(Ptorso, PhipR, 0.15); // Lado derecho de la pelvis

// Base de la cadera

Cilindro(PhipL, PhipR, 0.16);

// --- 3. EXTREMIDADES SUPERIORES (BRAZOS) ---

:Color(0.15, 0.35, 0.75); // Mangas de la playera

// Deltoides (Músculo del hombro)

DrawSphere(PshoulderL, 0.12);

DrawSphere(PshoulderR, 0.12);

:Color(0.85, 0.75, 0.65); // Tono de piel

// Bíceps / Tríceps

Cilindro(PshoulderL, PelbowL, 0.085);

Cilindro(PshoulderR, PelbowR, 0.085);

// Codos

DrawSphere(PelbowL, 0.075);

DrawSphere(PelbowR, 0.075);

// Antebrazos

Cilindro(PelbowL, PhandL, 0.065);

Cilindro(PelbowR, PhandR, 0.065);

// Manos

DrawSphere(PhandL, 0.06);

DrawSphere(PhandR, 0.06);

// --- 4. EXTREMIDADES INFERIORES (PIERNAS) ---

:Color(0.1, 0.2, 0.5); // Color del "pantalón"

//Articulación de la cadera

DrawSphere(PhipL, 0.14);

DrawSphere(PhipR, 0.14);

// Muslos

Cilindro(PhipL, PkneeL, 0.13);

Cilindro(PhipR, PkneeR, 0.13);

:Color(0.15, 0.25, 0.55); // Detalle de la rodilla

DrawSphere(PkneeL, 0.11);

DrawSphere(PkneeR, 0.11);

// Pantorrillas

Cilindro(PkneeL, PankleL, 0.095);

Cilindro(PkneeR, PankleR, 0.095);

// Tobillos

:Color(0.2, 0.2, 0.2);

DrawSphere(PankleL, 0.08);

DrawSphere(PankleR, 0.08);

// --- 5. PIES ---

:Color(0.15, 0.15, 0.15); // Color calzado

// Vector de ancho (X)

T_ancho = TranslatorN3(0.3, 0, 0);

// Vector de altura del empeine (Y)

T_alto = TranslatorN3(0, Hf, 0);


```

// Vector de talla (Z)
T_largo = TranslatorN3(0, 0, 0.6);

// Pie Izquierdo
D1_L = T_ancho * PankleL * (~T_ancho);
D2_L = T_alto * PankleL * (~T_alto);
D3_L = T_largo * PankleL * (~T_largo);
Box(PankleL, D1_L, D2_L, D3_L);

// Pie Derecho
D1_R = T_ancho * PankleR * (~T_ancho);
D2_R = T_alto * PankleR * (~T_alto);
D3_R = T_largo * PankleR * (~T_largo);
Box(PankleR, D1_R, D2_R, D3_R);

// =====
// ESCENARIO Y ENTORNO DEFINITIVO
// =====

// --- 1. Cielo (Color de fondo) ---
:Color(0.5, 0.65, 0.85); // Azul cielo limpio
DrawSphere(1 * e0, 30.0);

// --- 2. Suelo ---
:Color(0.45, 0.45, 0.45);

P_origen = 1 * e0;
T_suelo = TranslatorN3(0, -(Lp+Lc+2*Hf-BFy), 0);
P_origen_suelo = T_suelo * P_origen * (~T_suelo);

```

```
// Dimensiones de la plataforma (Largo, Ancho, Grosor)
T_suelo_ancho = TranslatorN3(15, 0, 0); // Extension hacia los lados (X)
T_suelo_largo = TranslatorN3(0, 0, 15); // Extension hacia adelante/atras (Z)
T_suelo_grosor = TranslatorN3(0, -0.02, 0);

D1_suelo = T_suelo_ancho * P_origen_suelo * (~T_suelo_ancho);
D2_suelo = T_suelo_largo * P_origen_suelo * (~T_suelo_largo);
D3_suelo = T_suelo_grosor * P_origen_suelo * (~T_suelo_grosor);

// Suelo
Box(P_origen_suelo, D1_suelo, D2_suelo, D3_suelo);
```

2. Modos de Operación y Control del Sistema

El sistema cuenta con una interfaz dual conmutada dinámicamente mediante herramientas de control de CLUViz (Checkboxes y Sliders unificados), permitiendo alternar de forma transparente entre las siguientes modalidades sin necesidad de interrumpir o modificar el código fuente:

A. Modo Manual (Interacción en Tiempo Real)

En este modo, el cuál es el que está activo de manera predeterminada, cada uno de los grados de libertad de las articulaciones principales (hombros, codos, caderas, rodillas y tobillos) se mapea a un control deslizante (*slider*) independiente. Los ángulos de rotación de los rotores locales están restringidos matemáticamente mediante límites anatómicos reales aproximados a la movilidad biológica de una persona promedio, evitando torsiones hiper extensivas incompatibles con el cuerpo humano.

B. Modo Animación (Interpolación por Archivo de Datos)

Al activar este modo, el flujo cinemático pasa a depender por completo de la lectura secuencial de matrices externas procedentes del archivo "datos.dat". El script procesa la información de forma secuencial sincronizando las columnas según la distribución geométrica especificada en "orden_datos.dat". Cada fila representa un fotograma clave

(*keyframe*) donde los motores de traslación global y rotación local de las uniones se actualizan de forma continua para generar una marcha suave y realista.

3. Implementación de la Cinemática Directa mediante Operadores de CGA

A diferencia del álgebra matricial clásica, este proyecto resuelve la cinemática directa mediante el uso de **Motores Conformes** aplicados recursivamente a través del "sándwich geométrico" $P = M * P * (\sim M)$. Esto previene de forma nativa fenómenos de indeterminación. La cadena cinemática se modela partiendo del centro de la cadera o base lumbar como raíz del árbol de transformaciones. Por ejemplo, para posicionar el hombro y el codo izquierdos se aplica de forma acumulativa:

1. **Transformación del Pecho:** Derivado de aplicar el motor de inclinación del torso sobre la traslación vertical del eje de la columna.
2. **Posición del Hombro:** Desplazamiento lateral rígido en el eje horizontal desde la posición del pecho.
3. **Posición del Codo:** Calculado aplicando el rotor tridimensional del hombro y extendiendo el bicep mediante el bi-vector conformal correspondiente, asegurando que la articulación siga fielmente el movimiento del torso.

4. Estrategias de Reutilización de Recursos y Optimización de Memoria

Con el objetivo de maximizar la eficiencia en el procesamiento gráfico dentro de CLUViz y promover el ahorro de recursos computacionales, se implementaron las siguientes estrategias:

- **Eliminación de Puntos Virtuales Desnormalizados:** Se erradicó el uso de trasladadores flotantes independientes (TranslatorN3) o promedios directos de multivectores para definir los centros del torso y la pelvis. Dichas operaciones alteraban el peso conforme en $e0$, forzando al procesador a re-normalizar constantemente las variables geométricas.
- **Modelado Basado en Conexión de Articulaciones Nativas:** Tanto el torso como la cadera se estructuraron de manera limpia utilizando cilindros y esferas que ligan directamente los puntos cinemáticos puros del esqueleto. Al reutilizar las mismas variables calculadas por la cinemática directa, se evita la creación y destrucción de

variables intermedias en la memoria en cada refresco de pantalla, asegurando una tasa de refresco estable durante la animación.

5. Diseño Estilizado del Humanoide y Modelado del Entorno

Para dar cumplimiento al requisito de un alto detalle visual en la representación del personaje, se diseñó una morfología balanceada mediante primitivas del álgebra conformal:

```

[ Cabeza: Esfera ]
|
[ Cuello: Cilindro ]
|
HombroIzq ----- Pecho ----- HombroDer <-- (Línea de Clavícula)
\           |           /
\   (Columna Central)   /   <-- (Costados del Torso)
\           |           /
\----- Cadera -----/   <-- (Línea de Cintura)
      /\
      / \
      /  \   <-- (Pelvis Triangular)
      /   \
CaderaIzq---CaderaDer

```

- **Torso y Pecho:** Se descartaron las macros de cubos desfasados debido a los artefactos visuales que introducen al rotar los planos en el infinito conformal. En su lugar, se configuró una caja torácica estilizada uniendo los hombros y la cintura con cilindros estructurales de diámetros variables, logrando una forma de trapecio anatómico inverso.
- **Pelvis:** Se modeló una estructura triangular rígida e inmune a las deformaciones animadas uniendo la base de la columna a las cabezas de los fémures nativos.
- **Escenario Seguro (Entorno de Simulación):** Para evitar errores de ejecución en tiempo de ejecución (*Runtime Errors*) vinculados a funciones de iluminación o fondos dependientes de librerías DLL del sistema, el entorno completo se modeló con geometría conforme nativa. El cielo se construyó mediante una esfera gigante con un

radio de 30 unidades que encierra por completo el espacio de simulación y tiñe el fondo de azul claro.



Conclusiones

A partir del diseño, desarrollo e implementación de la simulación cinemática del modelo humanoide tridimensional en el entorno CLUViz, se pueden obtener las siguientes:

- **Superioridad Matemática del Álgebra Geométrica Conformal (CGA):** El proyecto demostró empíricamente que la CGA es un marco matemático robusto para resolver problemas que a otras álgebras les tomaría una mayor cantidad de cálculos y recursos computacionales concretar de manera satisfactoria. La representación de las rotaciones locales y traslaciones globales mediante *motores conformales* en el espacio de Minkowski eliminó los problemas clásicos del álgebra matricial, permitiendo un flujo continuo y orgánico durante la animación.
- **Importancia de la Estabilidad Cinemática Pura:** Durante la fase de desarrollo, quedó evidenciado que el modelado tridimensional en CGA es altamente sensible a la manipulación directa de puntos flotantes. La erradicación de "puntos virtuales" calculados mediante operaciones no conformes (como traslaciones locales fijas o promedios directos) y su sustitución por una estructura que liga exclusivamente las variables articulares demostró ser la estrategia más eficiente para evitar artefactos visuales, colapsos y descoordinaciones geométricas al momento de ejecutar la animación en comparación a si cada elemento actuara de manera independiente, pero a la par del resto.
- **Versatilidad de la Interfaz Conmutada:** La implementación cumple tanto con la implementación del modo manual para manipular al personaje directamente como con el modo "animación" que ejecuta la secuencia a partir de la información obtenida del archivo "datos.dat". El **modo manual** validó el comportamiento individualizado y la física del esqueleto bajo límites angulares fieles a la biomecánica humana real, mientras que el **modo animación** probó la capacidad del script para procesar de forma externa, secuencial e ininterrumpida grandes volúmenes de datos, simulando una marcha continua estable.
- **Eficiencia y Optimización Gráfica:** Se concluye que la optimización en programación gráfica no solo radica en la renderización, sino en la reutilización de

recursos matemáticos. Al anclar los cilindros y volúmenes anatómicos a los mismos motores cinemáticos previamente calculados en lugar de exigir a la computadora la constante normalización de variables flotantes en cada fotograma, se optimizó de forma drástica la memoria, manteniendo una tasa de refresco fluida y estable.

- **Autonomía y Estabilidad del Entorno:** La necesidad de desarrollar un escenario envolvente empleando geometría conformal nativa demostró la importancia de crear scripts independientes del sistema que prevengan errores en tiempo de ejecución asociados a librerías externas. Este enfoque no solo garantizó la estabilidad del software, sino que enriqueció la experiencia visual al proveer una escala métrica de referencia indispensable para evaluar la velocidad y el avance en profundidad del humanoide.

En resumen, este proyecto final consolida la viabilidad de la Álgebra Geométrica Conformal como una herramienta de vanguardia para la ingeniería de software y la geometría computacional, ofreciendo un control cinemático más limpio, eficiente y tanto matemáticamente como computacionalmente más eficiente que los métodos geométricos tradicionales.

UNIVERSIDAD
Panamericana

Bibliografía:

- Hildenbrand, D. (2013). *Foundations of Geometric Algebra Computing*. Springer.
<https://doi.org/10.1007/978-3-642-31794-1>.

